NPTEL Software Testing (Advanced) Ultimate Study Guide & Master Prep Booklet

Comprehensive Theoretical Explanations, Deep Numerical Calculations with Formulas, Memory Tricks, Mnemonics, and 100 Practice MCQs

PART 1: Comprehensive Topic Breakdowns, Math Solutions, & Memory Mnemonics

Topic 1: Input Space Partitioning (ISP)

Input Space Partitioning (ISP) models the input domain of a program by grouping similar inputs into partitions or "equivalence classes." It assumes that all values inside a single partition trigger identical behavioral paths in the software, allowing us to test only a representative value rather than an infinite range.

The Partitioning Constraints

To ensure a mathematically sound model, any partition P of a domain D must satisfy:

- Completeness (Collectively Exhaustive): The union of all blocks must equal the entire domain (Union of b_i = D). No value is left behind.
- Disjointness (Mutually Exclusive): No two blocks may overlap (b_i Intersection b_j = Empty Set for i != j). No value can belong to multiple blocks simultaneously.

Designing Test Cases: ECC vs. BCC

- Each Choice Coverage (ECC): Every block in every partition must be represented in at least one test case.
- Base Choice Coverage (BCC): First, define a realistic, normal "Base Case." Then, construct subsequent test cases by shifting exactly one partition to a non-base block at a time while holding all other partitions at their base choices.

Numerical Application & Formula Example:

Consider a function calculateTax(age, income, residencyStatus) with the following partitions:

1. Age (3 blocks): A_1 = [0, 17], A_2 = [18, 64] (Base Choice), A_3 = [65+]
2. Income (4 blocks): I_1 = [<= 0], I_2 = [1, 50k] (Base Choice), I_3 = [50k+ - 150k], I_4 = [150k+]
3. Residency Status (2 blocks): R_1 = Resident (Base Choice), R_2 = Non-Resident

Formula for All-Combinations Coverage (ACoC):

$$\text{Total Test Cases} = \text{Product of } |P_i|$$

Applying the values:

$$\text{Total ACoC Test Cases} = 3 \text{ (Age)} \times 4 \text{ (Income)} \times 2 \text{ (Residency)} = 24 \text{ tests}$$

Formula for Base-Choice Coverage (BCC):

$$\text{Total Test Cases} = 1 + \text{Sum}(d_i - 1)$$

Applying the values:

$$\text{Total BCC Test Cases} = 1 + (3 - 1) + (4 - 1) + (2 - 1) = 1 + 2 + 3 + 1 = 7 \text{ tests}$$

Constructing the BCC Test Cases (Base case: $[A_2, I_2, R_1]$):

- Test 1 (Base Case): $[A_2, I_2, R_1]$
- Test 2 (Vary Partition 1): $[A_1, I_2, R_1]$
- Test 3 (Vary Partition 1): $[A_3, I_2, R_1]$
- Test 4 (Vary Partition 2): $[A_2, I_1, R_1]$
- Test 5 (Vary Partition 2): $[A_2, I_3, R_1]$
- Test 6 (Vary Partition 2): $[A_2, I_4, R_1]$
- Test 7 (Vary Partition 3): $[A_2, I_2, R_2]$

Memory Trick / Mnemonic:

"BCC = Base + One-at-a-Time"
Think of a luxury car customizer: You start with the standard base model (Base Case). To test each custom option, you change only the wheels (Test 2), only the paint (Test 3), or only the engine (Test 4). You never change two things at once, so you can isolate exactly what broke the design!

Topic 2: Graph Fundamentals & Structural Coverage

A Control Flow Graph (CFG) represents the execution flow of a system where Nodes are computational blocks and Edges are transfers of control.

Core Definitions

- Simple Path: A path where no node is visited more than once, with the single exception that the first and last node can be identical (allowing a clean loop).
- Prime Path: A simple path that is not a subpath of any other simple path. It is as long as a simple path can possibly get before looping or repeating.

Coverage Subsumption Hierarchy

- Prime Path Coverage (PPC) subsumes Edge-Pair Coverage (EPC).
- Edge-Pair Coverage (EPC) subsumes Edge Coverage (EC).
- Edge Coverage (EC) subsumes Node Coverage (NC).

Numerical Application & Step-by-Step Prime Path Calculation:

Let's find all the prime paths for the following graph:
- N = {1, 2, 3, 4}
- E = {(1,2), (2,3), (3,2), (2,4)}

Step 1: List all simple paths of length 0 (Nodes)
[1], [2], [3], [4]

Step 2: List all simple paths of length 1 (Edges)
[1, 2], [2, 3], [3, 2], [2, 4]

Step 3: List all simple paths of length 2
- [1, 2, 3]
- [1, 2, 4]
- [2, 3, 2] (Simple - starts and ends at 2, no internal node repetitions)
- [3, 2, 3] (Simple - starts and ends at 3, no internal node repetitions)
- [3, 2, 4]

Step 4: List all simple paths of length 3
- [1, 2, 3, 2] (NOT simple, because node 2 is repeated internally)
- [3, 2, 3, 2] (NOT simple, node 3 is repeated internally)

Step 5: Identify the Prime Paths (Paths that are not subpaths of any other simple path)
We look at our lists of simple paths and filter out those that are contained within longer ones:
1. [1, 2, 3] is contained in no longer simple path. (Prime Path 1)
2. [1, 2, 4] is contained in no longer simple path. (Prime Path 2)
3. [2, 3, 2] is contained in no longer simple path. (Prime Path 3)
4. [3, 2, 3] is contained in no longer simple path. (Prime Path 4)
5. [3, 2, 4] is contained in no longer simple path. (Prime Path 5)

Memory Trick / Mnemonic:

"Please Eat Every Nut" Prime Path Edge-Pair Edge Node.
Visually, think of traveling: Visiting every street corner (Node) is easy. Driving down every single road (Edge) is harder. Driving down every consecutive pair of streets (Edge-Pair) is even harder. Driving the entire maximum scenic route (Prime Path) is the ultimate coverage!

Topic 3: CFG and Source Code Mapping

To map actual code to a graph:

- Basic Block: A sequence of consecutive statements with one entry point and one exit point. No branching occurs inside. If statement 1 executes, all statements in the block are guaranteed to execute.
- Decisions: Loops (while, for) and conditions (if-else) split a single node into multiple exiting paths.

Memory Trick / Mnemonic:

"Statements are Towns, Branches are Roads."
You can visit every town (100% Statement Coverage) without driving down every road (Branch Coverage). For instance, if there is a highway bypassing a town directly, you only need one route to hit the town, leaving the alternate bypass unexplored. Therefore: Branch Coverage strictly subsumes Statement Coverage.

Topic 4: Data-Flow Testing

Data-flow testing tracks where variables are created, modified, and used.

Core Definitions

- Definition (def): An assignment of a value to a variable (e.g., x = 5).
- Use: A read access to a variable.
  - c-use (computational): Used inside a math equation or print statement (associated with Nodes).
  - p-use (predicate): Used to determine a branching condition (associated with Edges).
- Def-Clear Path: A path between a def and a use of a variable where the variable is never reassigned or modified.

Numerical Application & Step-by-Step du-pair analysis:

Consider the following program snippet:

```
1: x = 10;
2: if (x > 5)
3:    y = x + 2;
4: else
5:    y = 0;
6: print(y);
```

Let's find the def-clear paths and identify the du-pairs for variable x:
- Definitions of x: {1}
- Uses of x: {2} (p-use on edge (2,3) and (2,5)), {3} (c-use at node 3)

du-pairs for x:
1. (1, (2,3)) - Path: [1, 2] (Def-clear for x)
2. (1, (2,5)) - Path: [1, 2] (Def-clear for x)
3. (1, 3) - Path: [1, 2, 3] (Def-clear for x)

If we were to re-define x at node 2 (e.g., x = 4), the path [1, 2, 3] would no longer be def-clear for x's definition at node 1.

Memory Trick / Mnemonic:

"C-use is Calculation (on Nodes), P-use is Path-direction (on Edges)."
Think of a variable as a runner carrying a baton. The baton is handed over (def). The runner must run a clean path without dropping or swapping the baton (def-clear path) until they pass it to a calculator machine (c-use) or a fork in the road where they must choose a path based on its weight (p-use).

Topics 5, 6, & 7: Logic Coverage & Active/Inactive Clause Coverage

Logic criteria test complex boolean formulas.

- Predicate Coverage (PC): The overall predicate must evaluate to both True and False.
- Clause Coverage (CC): Every individual clause (variable) must evaluate to True and False. CC does NOT guarantee PC!

Numerical Application & Determination/RACC Formula:

Let's mathematically determine when clause A determines the predicate:

$$P = A \text{ AND } (B \text{ OR } C)$$

To find when A determines P, we apply the boolean difference formula:

$$P_A = P[A \leftarrow \text{True}] \text{ XOR } P[A \leftarrow \text{False}] = \text{True}$$

Let's substitute True and False for A in our predicate P:

- P[A <- True] = True AND (B OR C) = B OR C
- P[A <- False] = False AND (B OR C) = False

Now, XOR them together:

$$P_A = (B \text{ OR } C) \text{ XOR False} = B \text{ OR } C$$

For A to determine P, we need P_A = True, which means:

$$B \text{ OR } C = \text{True}$$

This means clause A determines the predicate P if and only if B is True, C is True, or both are True.

Constructing the RACC Test Cases for Major Clause A:

For RACC, the minor clauses (B, C) must hold the same values while A toggles between True and False. Let's pick B = True, C = False (which satisfies B OR C = True).

- Test Case 1 (A = True): [T, T, F] P evaluates to True
- Test Case 2 (A = False): [F, T, F] P evaluates to False

This pair of test cases perfectly satisfies RACC for clause A.

> Memory Trick / Mnemonic:
>
> "RACC = Rigid, CACC = Changed outcome, GACC = Go wild."
> In RACC, the minor clauses are Rigid (completely identical). This isolates the major clause perfectly like a scientific lab control experiment.
> In CACC, we care about the Changed outcome of the final predicate.
> In GACC, minor clauses can Go wild (change freely as long as the major clause determines the formula).

Topics 8 & 9: Mutation Testing & Mutation Score Calculations

Mutation testing is a fault-based technique where we introduce tiny bugs (mutants) into our code and check if our test suite can catch and kill them.

Numerical Application & Formula Example:

Let's calculate the Mutation Score for a program with the following parameters:
- Total Synthesized Mutants (M_T): 150
- Mutants Killed by Test Suite (M_K): 110
- Equivalent Mutants (M_E): 15

Formula for Mutation Score:

$$\text{Mutation Score} = (M_K / (M_T - M_E)) \times 100\%$$

Let's apply the numbers:

$$\text{Mutation Score} = (110 / (150 - 15)) \times 100\% = (110 / 135) \times 100\% \text{ approx. } 81.48\%$$

To achieve a 100% mutation score, the test suite must be expanded to kill the remaining 25 non-equivalent live mutants (135 - 110 = 25).

Memory Trick / Mnemonic:

"Kill the Bad, Spare the Twins (Equivalent)."
Your test suite is a security guard. The mutants are imposter clones. A strong guard spots and kills all imposters. But if an imposter is a perfect identical twin (equivalent mutant), even the best guard cannot distinguish them!

Topic 10: Specification & FSM Testing

- Finite State Machines (FSM): Systems are represented as states and transitions.
- OO Testing Issues:
  - Encapsulation: Must verify state changes do not bypass interface boundaries.
  - Inheritance & Polymorphism: Parent class tests must be rerun on subclass objects to ensure inherited methods do not corrupt new fields.

Memory Trick / Mnemonic:

"States are hotel rooms, Transitions are hallways."
State Coverage means entering every room. Transition Coverage means walking through every hallway. You cannot complete transition coverage without achieving state coverage first!

Topic 11: Regression Testing & APFD Calculations

Regression testing ensures that code updates or bug fixes do not break existing, unchanged features.
- Minimization: Permanently deletes duplicate tests.
- Selection: Temporarily selects tests relevant to the modified code.
- Prioritization: Reorders tests to find bugs as fast as possible. Measured using APFD (Average Percentage of Faults Detected).

Numerical Application & APFD Formula Example:

The APFD metric evaluates how quickly a prioritized test suite detects faults.

APFD Formula:

$$APFD = 1 - (\text{Sum of } T_i / (n \times m)) + 1 / (2n)$$

Where:
- n = total number of test cases
- m = total number of faults
- T_i = position of the first test case in the sequence that detects fault i

Let's calculate the APFD for a suite of 5 tests (n=5) designed to catch 3 faults (m=3):
Suppose the test sequence is [T_1, T_2, T_3, T_4, T_5] and:
- Fault 1 is caught first by T_2 T_1 = 2
- Fault 2 is caught first by T_1 T_2 = 1
- Fault 3 is caught first by T_4 T_3 = 4

Let's substitute these values into the formula:

$$APFD = 1 - ((2 + 1 + 4) / (5 \times 3)) + 1 / (2 \times 5)$$

Applying the math:

$$APFD = 1 - (7 / 15) + 1 / 10 = 1 - 0.4667 + 0.10 = 0.6333 \text{ approx. } 63.33\%$$

If we reorder the suite so that the first test case detects all faults, the APFD score increases toward 100%.

Memory Trick / Mnemonic:

"Minimization cleans the closet, Selection picks tonight's outfit, Prioritization decides which part of the outfit to wear first."
Minimization is permanent housekeeping. Selection is context-specific. Prioritization is order of execution.

Topics 12 & 13: Non-Functional & Web/Mobile Testing

- Load vs. Stress: Load is testing typical peak performance. Stress is pushing it until it snaps to see how it recovers.
- Mobile Interruption Testing: Verifies how the app handles sudden phone calls, SMS alerts, or network swaps (Wi-Fi to 5G).

Memory Trick / Mnemonic:

"Load is carrying heavy groceries. Stress is seeing how many bricks you can stack on your back before you collapse."

PART 2: 100 NPTEL Exam-Style Multiple Choice Questions

Input Space Partitioning (ISP)

Q1. To form a mathematically sound partition of an input domain, the blocks must be:

A. Complete and overlapping
B. Disjoint and incomplete
C. Complete and disjoint
D. Continuous and linear

Q2. If an input value x=10 belongs to two different blocks in the same partition, which property is violated?

A. Completeness
B. Disjointness
C. Subsumption
D. Determinism

Q3. Interface-based input space partitioning is primarily characterized by:

A. Examining parameters in isolation based on the function signature
B. Analyzing internal variables inside a loop
C. Mapping system states in a finite state machine
D. Evaluating mutation scores

Q4. Functionality-based input space partitioning is considered more advanced because it:

A. Does not require functional requirements
B. Incorporates semantic relationships and physical constraints between different parameters
C. Always produces exactly three test cases
D. Bypasses the disjointness rule

Q5. A function has three parameters with partition block sizes of 2, 3, and 5. What is the total number of test cases under All-Combinations Coverage (ACoC)?

A. 10
B. 15
C. 30
D. 45

Q6. For the partition block sizes in Q5 (2, 3, and 5), what is the minimum number of tests needed for Each-Choice Coverage (ECC)?

A. 2

B. 3

C. 5

D. 30

Q7. For the partition block sizes in Q5 (2, 3, and 5), what is the exact number of test cases needed for Base-Choice Coverage (BCC)?

A. 8

B. 10

C. 11

D. 30

Q8. Which coverage criterion is guaranteed to be fully covered if All-Combinations Coverage is achieved?

A. Only Each-Choice Coverage

B. Only Base-Choice Coverage

C. Both Each-Choice and Base-Choice Coverage

D. Neither

Q9. In Base-Choice Coverage, how are alternative test cases generated from the base case?

A. By altering all parameters to non-base values simultaneously

B. By altering exactly one parameter to a non-base block at a time while holding all others at their base values

C. By selecting random values from the entire domain

D. By setting all parameters to null

Q10. What is the primary drawback of All-Combinations Coverage (ACoC)?

A. It fails to test boundary values

B. It is highly susceptible to compiler warnings

C. The number of test cases scales exponentially with the number of parameters

D. It violates the disjointness property

Graph Fundamentals & Structural Coverage

Q11. In a directed graph representation of software, a path is simple if:

A. It contains only one node

B. No node is repeated, except possibly the first and last node

C. It contains no decision branches

D. It bypasses the final node

Q12. A simple path that is NOT a subpath of any other simple path is a:

A. Du-path
B. Prime Path
C. Sidetrip path
D. Complete path

Q13. In graph testing, Node Coverage is functionally identical to:

A. Branch Coverage
B. Statement Coverage
C. Path Coverage
D. Logic Coverage

Q14. Edge Coverage strictly subsumes which of the following criteria?

A. Edge-Pair Coverage
B. Node Coverage
C. Prime Path Coverage
D. Complete Path Coverage

Q15. If a test suite achieves 100% Edge-Pair Coverage, does it guarantee Edge Coverage?

A. Yes, because EPC strictly subsumes EC
B. No, EPC is weaker than EC
C. Only if the graph has no loops
D. Only if the graph has a single final node

Q16. What is the maximum length of a sidetrip in graph touring?

A. Always of length 1
B. It can be of any length as long as it starts and ends at the same node
C. It cannot contain edges
D. It must equal the graph's total node count

Q17. For a graph with nodes {1, 2, 3} and edges {(1,2), (2,3), (3,2)}, which of the following is a prime path?

A. [1, 2, 3]
B. [2, 3, 2]
C. [3, 2, 3]
D. All of the above

Q18. Can Prime Path Coverage be satisfied for a graph containing infinite loops?

A. No, because the path length is infinite
B. Yes, because the set of prime paths is always finite, even in the presence of loops
C. No, because loops block path execution
D. Only if we execute the loops exactly once

Q19. If a graph has nodes {1, 2, 3, 4} and edges {(1,2), (2,3), (3,4)}, how many prime paths exist?

A. 1
B. 3
C. 4
D. 6

Q20. For the graph in Q19, what is the longest prime path?

A. [1, 2]
B. [1, 2, 3]
C. [1, 2, 3, 4]
D. [2, 3, 4]

CFG & Source Code Mapping

Q21. A sequence of statements with a single entry point and a single exit point is called a:

A. Condition block
B. Basic block
C. Logic node
D. Loop header

Q22. In a Control Flow Graph, an if-else block without an explicit else statement:

A. Results in a single edge with no branching
B. Still results in a decision node branching into a true path and a false path bypass edge
C. Cannot be modeled as a CFG
D. Results in a self-loop

Q23. Which code block structure in a CFG represents a self-loop or back-edge?

A. Sequential variable assignments
B. An if-then-else statement
C. A while loop
D. A nested class declaration

Q24. If a program achieves 100% Statement Coverage, which of the following is true?

A. It must achieve 100% Branch Coverage

B. It may have missed empty else branches

C. It has tested all possible logic combinations

D. It has executed all prime paths

Q25. Branch Coverage is considered stronger than Statement Coverage because:

A. It requires writing fewer test cases

B. It evaluates all decision branches, including empty outcomes

C. It works strictly on external specifications

D. It does not require a CFG

Q26. What is the minimum number of test cases needed to achieve 100% Statement and Branch coverage for a single if-then-else block?

A. 1

B. 2

C. 3

D. 4

Q27. How is a nested if statement modeled in a CFG?

A. As a single node

B. As sequential decision nodes where one branch of the first decision leads directly to the second decision node

C. As an independent disjoint graph

D. It cannot be represented

Q28. In a CFG, the exit node of a method represents:

A. The main method declaration

B. The return point or end of execution of that method

C. A basic block with no inputs

D. A system crash point

Q29. Which of the following statements about basic blocks is true?

A. A basic block can contain multiple branch points

B. If the first statement of a basic block executes, all others in that block execute sequentially

C. A basic block always ends with a loop

D. Basic blocks can only contain variable declarations

Q30. Control Flow Graph testing is a form of:

A. Black-box testing
B. White-box testing
C. Performance testing
D. Regression testing

Data-Flow Testing

Q31. In data-flow testing, a variable is "defined" (def) when it is:

A. Read from a database
B. Assigned a value or instantiated
C. Out of scope
D. Passed as a constant argument

Q32. A "use" of a variable in data-flow testing is classified into:

A. Dynamic and static uses
B. Computational (c-use) and Predicate (p-use)
C. Primary and secondary uses
D. Input and output uses

Q33. Which of the following is a c-use of variable y?

A. if (y > 10)
B. x = y + 5;
C. while (y == true)
D. assert(y != null);

Q34. Which of the following is a p-use of variable y?

A. z = y * 2;
B. if (y)
C. print(y);
D. y = 10;

Q35. A path is "def-clear" with respect to variable x if:

A. x is defined at every node in the path
B. x is not redefined at any node or edge along the path
C. x is set to 0
D. The path contains no loops

Q36. What is a "du-pair"?

A. A dual-use variable

B. A pair consisting of a definition of variable x and its subsequent use, connected by a def-clear path

C. Two definitions of the same variable in a basic block

D. A duplicate variable name

Q37. Which of the following data-flow criteria is the strongest?

A. All-Defs Coverage

B. All-Uses Coverage

C. All-du-paths Coverage

D. All-p-uses Coverage

Q38. All-Defs Coverage requires exercising at least one def-clear path from each definition to:

A. Every single use of that variable

B. At least one use of that variable

C. The exit node of the program

D. The entry node of the program

Q39. If variable x is defined at node 1, and c-uses of x exist at nodes 2 and 3, how many du-pairs are there (assuming def-clear paths exist)?

A. 1

B. 2

C. 3

D. 4

Q40. Tracking variables from their creation to their consumption is the core principle of:

A. Logic testing

B. Data-flow testing

C. Mutation testing

D. Non-functional testing

Logic Coverage

Q41. For a predicate P = A AND B AND C, how many test cases are required for Combinatorial Coverage (CoC)?

A. 3

B. 6

C. 8

D. 12

Q42. For the predicate P = A OR B, which of the following test suites satisfies Predicate Coverage (PC)?

A. {TT, FF}
B. {TF, FT}
C. {TT, TF}
D. Both A and B satisfy PC

Q43. Which of the following statements is true about Clause Coverage (CC)?

A. It guarantees Predicate Coverage
B. It requires all individual clauses to evaluate to both True and False
C. It is a black-box partition technique
D. It only applies to numeric variables

Q44. What is a "predicate" in logic coverage?

A. A single boolean variable
B. A boolean expression that evaluates to True or False
C. A function with no parameters
D. A variable assignment statement

Q45. Why is Combinatorial Coverage (CoC) rarely used in industrial systems with large boolean conditions?

A. It is mathematically invalid
B. The number of test cases scales exponentially (2^n) with the number of clauses
C. It is weaker than Predicate Coverage
D. It does not support logical AND

Q46. In the predicate P = (A AND B) OR C, how many distinct clauses are there?

A. 1
B. 2
C. 3
D. 4

Q47. If a test suite achieves 100% Predicate Coverage, does it guarantee 100% Clause Coverage?

A. Yes, always
B. No, because clauses may remain untested in their alternate states
C. Only if the predicate has 1 clause
D. Only if there are no OR operators

Q48. Which logic coverage criterion evaluates every row of a predicate's truth table?

A. Predicate Coverage
B. Clause Coverage
C. Combinatorial Coverage
D. Active Clause Coverage

Q49. In logic coverage, if a predicate has n independent clauses, how many rows are in its truth table?

A. n
B. 2^n
C. n^2
D. 2n

Q50. If we have a predicate with 5 clauses, what is the size of its complete truth table?

A. 5
B. 10
C. 25
D. 32

Active & Inactive Clause Coverage (ACC / ICC)

Q51. A major clause c "determines" a predicate P if and only if:

A. Changing c's value changes the value of P, keeping all other clauses constant
B. c is always True
C. P is always False
D. All minor clauses are True

Q52. The determination condition can be calculated using which logical operation?

A. AND
B. OR
C. XOR
D. NOT

Q53. Under General Active Clause Coverage (GACC), minor clauses:

A. Must be identical in both tests for the major clause
B. Do not have to be identical in both tests as long as the major clause determines the predicate in both
C. Must all be False
D. Must all be True

Q54. Under Restricted Active Clause Coverage (RACC), the minor clauses:

A. Can change values freely
B. Must remain completely identical in both the True and False test cases for the major clause
C. Must be set to null
D. Must evaluate to the opposite of the major clause

Q55. In Correlated Active Clause Coverage (CACC), the final predicate outcomes for the two tests must:

A. Both be True
B. Both be False
C. Toggle (one must be True and the other must be False)
D. Be completely ignored

Q56. For the predicate P = A AND B, what value must minor clause B take for major clause A to determine P?

A. False
B. True
C. Either True or False
D. None of the above

Q57. For the predicate P = A OR B, what value must minor clause B take for major clause A to determine P?

A. False
B. True
C. Either True or False
D. None of the above

Q58. In Inactive Clause Coverage (ICC), the major clause is set up such that:

A. It determines the predicate
B. It does NOT determine the predicate, and the predicate outcome remains constant regardless of the major clause
C. It is always set to True
D. It is removed from the expression

Q59. GICC differs from RICC because RICC requires:

A. Fewer test cases
B. Minor clauses to remain completely identical in both tests
C. The major clause to be active
D. No testing of boolean expressions

Q60. Which criterion provides the most rigorous, isolated examination of a major clause's logic?

A. GACC
B. CACC
C. RACC
D. GICC

Mutation Testing

Q61. What is a "mutant" in mutation testing?

A. A program with compilation errors
B. A modified copy of the source code containing a single intentional bug
C. A high-performance optimization of the program
D. An automated test script

Q62. A mutant is "killed" when:

A. It causes the compiler to crash
B. At least one test case produces a different output on the mutant compared to the original program
C. It is manually marked as resolved by a developer
D. It is deleted from the server

Q63. A mutant that behaves exactly like the original program under all possible inputs is called an:

A. Active mutant
B. Equivalent mutant
C. Indestructible mutant
D. Static mutant

Q64. Detecting equivalent mutants is a problem that is:

A. Solvable in constant time
B. Mathematically undecidable
C. Solvable only in C++
D. Already automated in all IDEs

Q65. If a test suite kills 90 mutants out of 100, and 5 mutants are equivalent, what is the mutation score?

A. 90%
B. 94.74%
C. 95%
D. 85%

Q66. Which of the following is a classic mutation operator?

A. Swapping < to <=
B. Deleting a statement
C. Changing + to -
D. All of the above

Q67. A "live" mutant indicates that:

A. The test suite is strong
B. The test suite has a gap and failed to detect the introduced bug
C. The code has compilation errors
D. The program executed too fast

Q68. Under Strong Mutation, a mutant is killed only if:

A. The mutated statement is executed
B. The mutation affects the internal state and propagates a different output to the program termination
C. The test suite has no boundary cases
D. The compiler displays a warning

Q69. Under Weak Mutation, a mutant is killed if:

A. The mutated statement is executed and a state difference is observed immediately after execution
B. The program terminates successfully
C. The database is updated
D. No compilation is needed

Q70. Mutation testing is primarily used to evaluate:

A. Production code speed
B. The effectiveness and thoroughness of a test suite
C. Compiler optimization
D. Developer coding style

Integration & OO Mutation

Q71. OO Mutation operators focus primarily on mutating:

A. Arithmetic expressions inside private methods
B. Class relationships, inheritance, access visibility, and polymorphic bindings
C. Server configurations
D. SQL query syntax

Q72. Which of the following is a valid OO mutation operator?

A. Deleting a call to super()
B. Swapping class instantiation dynamic types
C. Changing a variable from private to public
D. All of the above

Q73. A "Polymorphic Shift" mutation operator changes:

A. The entire class hierarchy
B. The dynamic type binding of an object to a compatible sibling or parent class
C. The compiler target configuration
D. A loop condition

Q74. Mutating access modifiers (e.g., public to private) is highly useful for testing:

A. Database transaction isolation
B. Class encapsulation and interface visibility robustness
C. Program execution times
D. Compiler warning counts

Q75. Why are traditional mutation operators insufficient for OO programs?

A. OO programs do not use math operators
B. Traditional operators fail to target OO-specific structures like inheritance, encapsulation, and polymorphism
C. Traditional mutation is too slow
D. Traditional mutation is deprecated

Specification, FSM & OO Testing

Q76. Finite State Machine (FSM) testing represents a system using:

A. Sequential basic blocks
B. States, transitions, events, and actions
C. Mutation scores
D. Database tables

Q77. State Coverage in FSM testing requires that our test cases:

A. Execute every line of code
B. Visit every logical state in the machine at least once
C. Test all variables
D. End in the initial state

Q78. Transition Coverage in FSM testing requires that:

A. Every transition path between states is exercised at least once
B. The machine is permanently locked in one state
C. Tests bypass intermediate states
D. All variables are mutated

Q79. Object-Oriented testing must address "Polymorphism" by:

A. Only testing the parent class
B. Testing all possible dynamic bindings of polymorphic methods in subclasses
C. Making all methods static
D. Removing inheritance

Q80. In OO testing, encapsulation is tested by verifying that:

A. Objects can directly access each other's private fields
B. Object states can only be altered through their public interfaces and methods
C. Classes contain no variables
D. All variables are public

Regression Testing

Q81. Regression testing is conducted to:

A. Ensure old, working features are not broken by recent code changes or fixes
B. Find syntax errors in new code
C. Measure server response times
D. Minimize code files

Q82. Test Suite Minimization (Reduction) is a:

A. Temporary filter of test cases for a single commit
B. Permanent elimination of redundant test cases from the suite
C. Manual process of deleting comments
D. Compiler optimization level

Q83. Test Case Selection:

A. Permanently deletes tests
B. Selects a temporary subset of tests that are relevant to the modified parts of the software
C. Randomly executes half of the test suite
D. Requires rewriting the functional specifications

Q84. What is the main objective of Test Case Prioritization?

A. To reduce the test suite size
B. To order test execution so that high-benefit tests (like those with faster fault detection) run first
C. To run all tests in parallel
D. To test only the main class

Q85. Which metric is used to evaluate the quality of a prioritized test suite's fault detection rate?

A. Cyclomatic Complexity
B. Mutation Score
C. APFD (Average Percentage of Faults Detected)
D. Code Coverage

Q86. An APFD value ranges between:

A. -1 and +1
B. 0 and 1 (or 0% and 100%)
C. 1 and 10
D. -infinity and +infinity

Non-Functional Testing

Q87. Non-functional testing evaluates:

A. What features the software has
B. How well the system performs under constraint boundaries, workloads, and environments
C. The syntactic correctness of variable names
D. Class diagram architectures

Q88. Which of the following is a type of non-functional testing?

A. Node coverage testing
B. Stress testing
C. Data-flow testing
D. Mutation testing

Q89. Load Testing evaluates system behavior under:

A. Zero active users
B. Anticipated, typical, or peak workloads
C. Catastrophic server failure conditions
D. Invalid syntax settings

Q90. Pushing a system past its operational limit until it breaks is called:

A. Load Testing
B. Stress Testing
C. Performance Testing
D. Integration Testing

Q91. Security testing is primarily concerned with:

A. Checking memory leaks
B. Spotting vulnerabilities, risks, and protecting data from unauthorized access
C. Improving execution speed
D. Minimizing test suite sizes

Q92. Usability testing measures:

A. The number of system features utilized
B. How easily and intuitively a human can navigate and use the application interface
C. The database indexing speed
D. Code coverage metrics

Web & Mobile Application Testing

Q93. Web application testing must address which of the following?

A. Device battery drainage only
B. Browser rendering engine compatibility, responsive design, and network latency variations
C. Strict assembly instructions
D. Hardware camera calibration

Q94. Responsive Web Design testing ensures that the user interface:

A. Responds to mouse clicks in under 1 second
B. Adjusts and renders correctly on various screen resolutions, mobile devices, and browsers
C. Is written strictly in HTML5
D. Bypasses the server entirely

Q95. What is a key challenge unique to Mobile Application Testing?

A. Lack of operating systems
B. High environmental volatility such as network drops, physical interruptions, and low-resource constraints
C. Inability to use black-box techniques
D. Lack of variables

Q96. Interruption Testing in mobile apps verifies how the app behaves during:

A. Sudden incoming phone calls, SMS notifications, or low battery alerts
B. Server side restarts
C. Code modification during execution
D. Database query transactions

Q97. Testing a mobile application as it switches from Wi-Fi to a 5G network is an example of:

A. Load testing
B. Network swapping (connectivity state transition) testing
C. Regression testing
D. Spec-based testing

Q98. Resource consumption testing for mobile apps tracks:

A. The financial cost of cloud services
B. Battery drain, RAM utilization, and local storage consumption
C. The number of active testers
D. The compiler storage footprint

Q99. Running a mobile app on virtual environments mimicking real phone operating systems on a PC is called:

A. Emulator/Simulator testing
B. Physical device testing
C. Static analysis
D. Field testing

Q100. Compatibility testing for mobile applications checks if:

A. The code works on different OS versions (e.g., iOS 15 vs. iOS 16) and device profiles
B. The developer likes the user interface
C. The database is compatible with Oracle only
D. The app runs without an internet connection

PART 3: Official Answer Key

Question	Answer	Question	Answer	Question	Answer	Question	Answer
Q1	C	Q26	B	Q51	B	Q76	B
Q2	B	Q27	B	Q52	B	Q77	B
Q3	C	Q28	B	Q53	A	Q78	A
Q4	B	Q29	B	Q54	B	Q79	B
Q5	C	Q30	B	Q55	A	Q80	B
Q6	C	Q31	B	Q56	B	Q81	B
Q7	C	Q32	B	Q57	B	Q82	B
Q8	C	Q33	B	Q58	B	Q83	A
Q9	B	Q34	B	Q59	B	Q84	B
Q10	C	Q35	B	Q60	A	Q85	B
Q11	C	Q36	B	Q61	B	Q86	B
Q12	B	Q37	C	Q62	B	Q87	B
Q13	B	Q38	B	Q63	B	Q88	B
Q14	B	Q39	B	Q64	B	Q89	B
Q15	A	Q40	B	Q65	B	Q90	B
Q16	B	Q41	B	Q66	B	Q91	B
Q17	B	Q42	C	Q67	D	Q92	B
Q18	B	Q43	B	Q68	A	Q93	B
Q19	C	Q44	C	Q69	B	Q94	B
Q20	C	Q45	B	Q70	A	Q95	B
Q21	B	Q46	B	Q71	B	Q96	B
Q22	B	Q47	B	Q72	D	Q97	B
Q23	C	Q48	B	Q73	B	Q98	A
Q24	C	Q49	B	Q74	B	Q99	B
Q25	A	Q50	C	Q75	A	Q100	A